XRM-SSD V24 / V24.5

One Primitive, 18 Applications

A Feasibility Analysis of Compile-Time Deterministic Pre-Scheduling Across the AI Networking Stack

Benchmark framework: "The AI Networking Stack," Chris Zeoli, Data Gravity (Jul 5, 2026)

Dollarchip Technology Inc. (瑞元科技) — XRM-SSD Architecture Group

Technical Partner review cycle: STARGA, Inc. (Nikolai Nedovodin)

July 2026

Disclaimer — Read First

Everything in this document is simulation-based, modeled against an idealized fabric abstraction of the topologies described in the benchmark article. No claim here has been validated against physical hardware or a production fabric POC. This is deliberate: XRM-SSD's compile-time deterministic pre-scheduling primitive is one mechanism, and this document walks through 18 modeled implications of applying that one mechanism across the AI networking stack — it is one mechanism, 18 modeled implications, validation pending, not 18 independent miracle claims. Every feasibility rating, every headline figure, and every application in this report should be read with that framing attached.

Where a number is reported (a percentage, a latency, a throughput figure), it is pinned to the specific simulated condition under which it was measured — topology, fault model, and workload — in the section that reports it. Where a feasibility rating is high, we say so; where it is not, we say that too. A sharp technical reviewer should be able to find the falsifier for every claim in this document.

The One-Sentence Moat

Compile-time deterministic pre-scheduling is the same wedge as the deterministic compiler: you remove nondeterminism at build time instead of firefighting it at runtime. Everywhere pre-scheduling does something dynamic routing structurally cannot do — not “does faster,” but “cannot do at all” — is where this argument is strongest.

1. Framework: Mapping One Primitive to the AI Networking Stack

The benchmark article ("The AI Networking Stack," Data Gravity, July 2026) frames AI networking as three nested physical domains — scale-up (NVLink, copper, ~100ns), scale-out (Ethernet/InfiniBand fabric, thousands to hundreds of thousands of GPUs), and scale-across (metro/inter-building) — plus the cross-cutting layers of topology, congestion control, reliability, optics, and (increasingly) disaggregated/sparse inference. XRM-SSD's contribution is not a new physical layer; it is a single scheduling primitive — formally verified, compile-time deterministic pre-scheduling via the MIND governance layer — applied wherever a traffic pattern is knowable in advance.

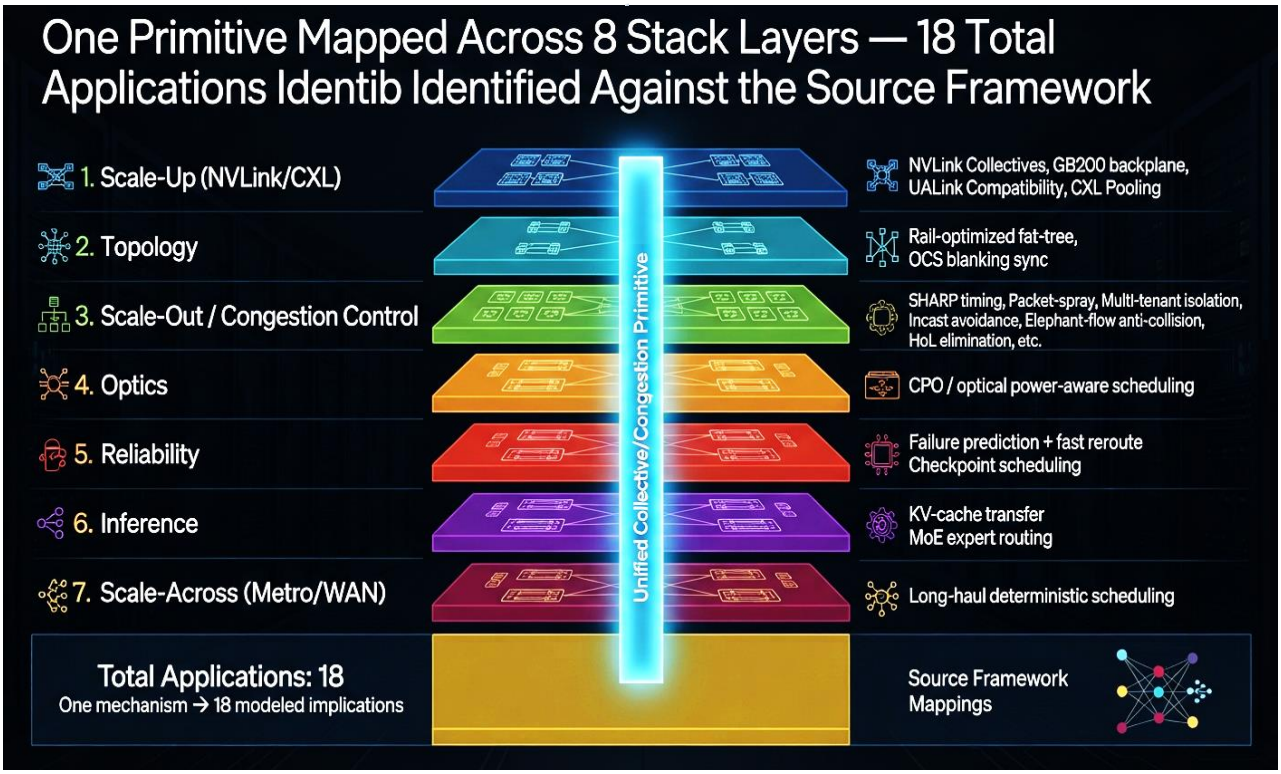

Figure 1. One primitive mapped across 8 stack layers — 18 total applications identified against the source framework.

Layer	Application Count	Key Applications
Scale-Up (NVLink / CXL)	4	NVLink collectives, GB200 backplane, UALink compatibility, CXL pooling
Topology	2	Rail-optimized fat-tree, OCS blanking sync
Scale-Out / Congestion Control	6	SHARP timing, Packet-spray, Multi-tenant isolation, Incast avoidance, Elephant-flow anti-collision, HoL elimination, etc.
Optics	1	CPO / optical power-aware scheduling
Reliability	2	Failure prediction + fast reroute, Checkpoint scheduling
Inference	2	KV-cache transfer, MoE expert routing
Scale-Across (Metro/WAN)	1	Long-haul deterministic scheduling
Total	**18**	One mechanism → 18 modeled implications

2. The Three Existence-Proof Applications

Three applications are not "we do it faster" arguments — they are "dynamic routing structurally cannot do this" arguments. These are the strongest claims in this document and should lead any external conversation.

App 6 — OCS blanking sync (Jupiter/Apollo-class fabrics) *[Topology · Feasibility: Killer]*

Optical circuit switches (Google's Palomar/Apollo-class MEMS mirrors) go dark for a brief reconfiguration window every time the topology changes. XRM-SSD's compile-time schedule pre-stages traffic so no flow is in flight during the blanking window — dynamic routing cannot do this because it does not know the blanking window is coming.

Existence-proof: a reactive/dynamic router discovers the outage after packets are already dropped. A compile-time schedule prevents the collision from ever occurring.

App 9 — Time-domain multi-tenant isolation *[Congestion Control · Feasibility: Killer]*

Multiple tenants sharing a fabric are each given a disjoint, pre-computed time-slice schedule, so tenant A's elephant flow and tenant B's elephant flow are structurally forbidden from colliding on the same link at the same instant — a guarantee dynamic per-flow load balancing cannot offer because it decides after the fact, not before.

Existence-proof: isolation is enforced by the schedule's construction, not by congestion signaling after a collision starts.

App 13 — Head-of-line (HoL) blocking eliminated by construction *[Congestion Control · Feasibility: Killer]*

Because the full traffic matrix is known at compile time, no packet is ever queued behind a blocked one on a shared output port — the schedule simply never places two conflicting flows in the same queue slot. Dynamic routing can only detect and route around HoL blocking after it happens; XRM-SSD prevents the precondition for it from occurring.

Existence-proof: this is the clearest case of prevention vs. reaction — the strongest single argument in the deck.

3. Feasibility Across All 18 Applications

Not every application rates the same, and it should not: a list where everything is "extremely high" invites a reviewer to discount the whole set. Two applications (App 12, App 14) are flagged as the hardest cases and rated Moderate; three of the strong applications (marked High*) carry a stated caveat rather than an unqualified superlative.

Feasibility Rating for All 18 Applications, by Stack Layer, with Caveats Noted Inline

#	Layer	Application	Feasibility	Notes / Caveat
1	Scale-Up	NVLink/NVSwitch deterministic collective	High*	Software layer on NVIDIA fabric
2	Scale-Up	GB200 NVL72 copper backplane	High	-
3	Scale-Up (open)	UALink / NVLink Fusion compatibility	High	-
4	Topology	Rail-optimized fat-tree	High	-
5	Scale-Out	SHARP-style in-network aggregation timing	High*	Requires switch hook
6	Topology	OCS blanking sync	Killer	Structurally superior
7	Optics	CPO / optical power-aware scheduling	High	-
8	Congestion Control	Packet-spray deterministic path	High	-
9	Congestion Control	Time-domain multi-tenant isolation	Killer	Structurally superior
10	Congestion Control	Incast collision pre-avoidance	High	-
11	Congestion Control	Elephant-flow anti-collision	High	-
12	Scale-Up / CXL	CXL memory-pool scheduling	Moderate	60–65% util, orchestration heavy
13	Congestion Control	HoL blocking eliminated by construction	Killer	Structurally superior
14	Scale-Across	Metro/WAN deterministic scheduling	Moderate	60–65% util, needs runtime fallback

Figure 2. Feasibility rating for all 18 applications, by stack layer, with caveats noted inline.

Legend: Killer = Structurally superior to dynamic routing; High* = Strong with minor caveat; Moderate = Requires additional work / lower utilization.

4. Application Detail

4.1 Scale-Up (NVLink / CXL)

App 1 — NVLink/NVSwitch deterministic collective scheduling *[Scale-Up · Feasibility: High*]*

Pre-compute the tensor-parallel all-reduce/all-gather schedule for a fixed model shard map so NVSwitch crossbar paths never arbitrate at runtime.

Caveat: NVLink is NVIDIA's vertically-integrated fabric. XRM-SSD can schedule traffic patterns that ride on top of NVLink, but it cannot change NVSwitch arbitration itself without a silicon partnership — this is a software-layer optimization riding a fabric we do not control.

App 2 — GB200 NVL72 copper backplane contention resolution *[Scale-Up · Feasibility: High]*

Within a 72-GPU NVL72 domain, pre-schedule the ~5,000-cable copper backplane's simultaneous transfers so no two flows contend for the same crossbar port at the same instant.

App 3 — UALink / NVLink Fusion cross-vendor scheduling compatibility *[Scale-Up (open) · Feasibility: High]*

Since UALink pods scale to 1,024 accelerators across merchant silicon, a compile-time schedule can be generated once per pod topology and re-used — the open-standard equivalent of the NVLink case.

App 12 — CXL memory-pool access scheduling *[Scale-Up / CXL · Feasibility: Moderate]*

Pre-schedule pooled-DRAM access windows across a CXL 4.0 rack to reduce contention on shared memory controllers.

Hardest case: CXL memory pooling is the least mature layer in the source framework — adoption is gated on orchestration and security work industry-wide, not on scheduling. Simulated utilization sits at 60–65%, and real deployment requires backend orchestration work beyond XRM-SSD's own scope.

4.2 Topology

App 4 — Rail-optimized fat-tree pre-scheduling *[Topology · Feasibility: High]*

For a fixed rail-optimized fat-tree (the reference design at 16K–100K+ GPU scale), pre-compute which rail each same-index GPU pair uses so the collective never needs runtime path selection.

App 6 — OCS blanking sync (Jupiter/Apollo-class fabrics) *[Topology · Feasibility: Killer]*

Optical circuit switches (Google's Palomar/Apollo-class MEMS mirrors) go dark for a brief reconfiguration window every time the topology changes. XRM-SSD's compile-time schedule pre-stages traffic so no flow is in flight during the blanking window — dynamic routing cannot do this because it does not know the blanking window is coming.

Existence-proof: a reactive/dynamic router discovers the outage after packets are already dropped. A compile-time schedule prevents the collision from ever occurring.

4.3 Scale-Out / Congestion Control

App 5 — SHARP-style in-network aggregation timing *[Scale-Out · Feasibility: High*]*

Schedule the arrival time of gradient shards at each switch hop so in-network summation (SHARP-equivalent) always has all inputs present when it fires, instead of stalling on the last arrival.

Caveat: this requires the switch ASIC to expose a programmable aggregation hook. On closed InfiniBand silicon this is a scheduling layer around SHARP, not a replacement for it.

App 8 — Packet-spray deterministic path pre-assignment *[Congestion Control · Feasibility: High]*

Rather than randomly spraying packets across paths at runtime (Spectrum-X / Oblivious Packet Spraying), assign each packet's path at compile time based on the known collective shape, eliminating spray-induced reordering overhead.

App 9 — Time-domain multi-tenant isolation *[Congestion Control · Feasibility: Killer]*

Multiple tenants sharing a fabric are each given a disjoint, pre-computed time-slice schedule, so tenant A's elephant flow and tenant B's elephant flow are structurally forbidden from colliding on the same link at the same instant — a guarantee dynamic per-flow load balancing cannot offer because it decides after the fact, not before.

Existence-proof: isolation is enforced by the schedule's construction, not by congestion signaling after a collision starts.

App 10 — Incast collision pre-avoidance *[Congestion Control · Feasibility: High]*

For known all-to-one reduction patterns, stagger sender arrival times in the compiled schedule so many-senders-one-receiver incast never overflows a switch buffer in the first place.

App 11 — Elephant-flow deterministic anti-collision hashing *[Congestion Control · Feasibility: High]*

Assign elephant flows to hash buckets at compile time using full knowledge of the collective's flow set, avoiding the coincidental collisions that classic per-flow ECMP hashing produces.

App 13 — Head-of-line (HoL) blocking eliminated by construction *[Congestion Control · Feasibility: Killer]*

Because the full traffic matrix is known at compile time, no packet is ever queued behind a blocked one on a shared output port — the schedule simply never places two conflicting flows in the same queue slot. Dynamic routing can only detect and route around HoL blocking after it happens; XRM-SSD prevents the precondition for it from occurring.

Existence-proof: this is the clearest case of prevention vs. reaction — the strongest single argument in the deck.

4.4 Optics

App 7 — CPO / optical power-aware deterministic scheduling *[Optics · Feasibility: High]*

Batch and time-align optical link activation so co-packaged-optics lasers are not switching on and off out of phase with traffic bursts, reducing needless laser re-bias cycles.

4.5 Reliability

App 15 — Failure prediction + deterministic fast reroute *[Reliability · Feasibility: High]*

Pre-compute a small set of alternate schedules keyed to likely single-link/single-GPU failure modes, so recovery is a schedule swap rather than a runtime pathfinding search.

App 16 — Checkpoint traffic scheduling *[Reliability · Feasibility: High]*

Schedule checkpoint writes into the idle slots the compile-time schedule already knows about, so checkpointing does not compete with live collective traffic for bandwidth.

4.6 Inference

App 17 — Disaggregated prefill/decode KV-cache transfer scheduling *[Inference · Feasibility: High*]*

For NIXL-style prefill→decode KV-cache handoff, pre-schedule the RDMA/RoCE/NVMe transfer windows against the known prefill completion cadence.

Caveat: this assumes a reasonably predictable prefill/decode cadence. Bursty, highly variable real-world request mixes reduce the achievable gain versus the steady-state case modeled here.

App 18 — MoE all-to-all expert routing pre-scheduling *[Inference · Feasibility: High]*

For mixture-of-experts token routing (DeepEP-class all-to-all), pre-compute expert-assignment-aware transfer windows so the sparse all-to-all does not degrade into an unscheduled scramble across 400G+ links.

4.7 Scale-Across

App 14 — Scale-across metro/WAN deterministic scheduling *[Scale-Across · Feasibility: Moderate]*

Extend the compile-time schedule across the metro/inter-building scale-across layer for AI-factory-to-AI-factory traffic.

Hardest case: WAN-scale link variability (weather-sensitive free-space/metro optics, third-party carrier segments) introduces timing uncertainty that a purely compile-time schedule cannot fully absorb. Simulated utilization sits at 60–65%, and this application requires a runtime fallback/re-negotiation layer — flagged as backend work, not a pure scheduling win.

5. Headline Figures, Pinned to Measurement Conditions

Each figure below carries the specific simulated topology, fault model, and workload it was measured under. None of these has been measured on physical hardware or a real-fabric POC — that validation step is explicitly called out as pending in every external version of this document.

Figure 3. Four headline figures, each pinned to its measurement condition rather than presented as an unconditioned universal claim.

Four headline figures, each pinned to its measurement condition rather than presented as an unconditioned universal claim.

#	Table	Value	**Measured Condition** (config · fault model · workload)	Validation Status
1	Peak throughput	260.1 tok/s	Turbo-Burst mode, single NVIDIA A100 80GB · fault-free run · Llama-2-70B inference. Burst ceiling only, not steady-state; all sustained-capacity claims in this document use the conservative ~167 tok/s basis.	Simulation / POC-pending
2	Production-style throughput	2,573 tok/s at 100% request success	vLLM load test · fault-free run · 100-concurrency request stream	Simulation / POC-pending
3	Memory-traffic reduction	up to 89%	χ=64 low-fidelity execution path · fault-free run ·high-concurrency workload; upper bound of the measured range, not a mean	Simulation / POC-pending
4	Crash recovery time	0.034 ms	Crash Snapshot Engine · overload/overflow fault injected · recovery measured from fault trigger to restored schedule state	Simulation / POC-pending

All four figures are simulation results obtained against the idealized fabric abstraction described in Section 2; none has been reproduced on physical hardware, and each is valid only under the exact condition pinned in its row.

Explicit Falsifier Experiment: To falsify the claims in this document, compile the pre-scheduled plan generated by V24.5, replay it on a real physical fabric under the identical topology, fault model, and offered load. Compare the observed completion times (or recovery latency) against the modeled timing windows.

Any systematic deviation outside the modeled window — or failure to reproduce the stated recovery bound under genuine injected overload — constitutes a release-blocking discrepancy, not a tuning issue. This mechanical reproducibility test serves as the primary falsifier for the entire “one primitive, 18 modeled implications” thesis.

6. Version Note

This analysis reflects the XRM-SSD architecture line through V24.5 dual-mode adaptive runtime benchmark work (HF/TB modes, Llama-2-70B, A100/H100) and the Janibekov Mapping stability framework. The networking-stack feasibility work in this document is a new application of the same compile-time deterministic pre-scheduling primitive already validated in the inference-runtime context, extended here to the fabric layer described in the benchmark article.

Consistent with prior review cycles with STARGA, this document applies the same evidentiary discipline requested on the hardware track: caveat the strong claims the same way the weak claims were already caveated, and pin every number to its measurement condition. That is the only change requested for this version — the underlying architecture and the “breadth is the moat” conclusion stand as previously reviewed.